

Data Parallel Computing

Олександр Ковальов, IM-51мн

«Київський політехнічний інститут імені Ігоря Сікорського»
Кафедра обчислювальної техніки

1 травня 2026 р.

Вступ та поняття паралелізму даних

Паралелізм даних дозволяє досягти високої продуктивності шляхом одночасного виконання однакових операцій над великими масивами. Програма розбивається на тисячі ідентичних потоків, кожен з яких оперує своєю частиною даних. Це дозволяє масштабувати алгоритми відповідно до кількості ядер GPU.

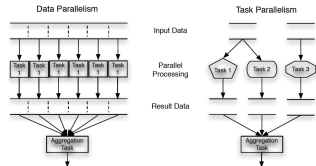
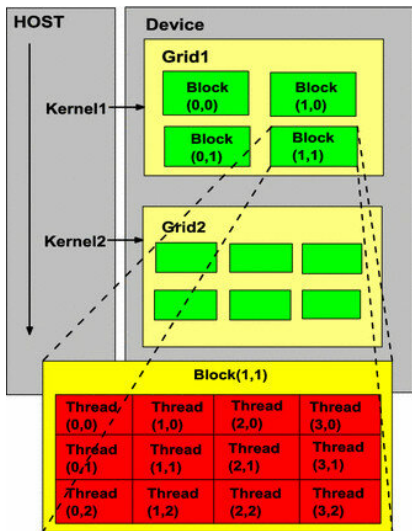


Рис. 1: Паралелізм задач vs даних

Архітектура гетерогенних обчислень CUDA



CUDA об'єднує CPU (хост) та GPU (пристрій). Хост керує логікою та пам'яттю, а пристрій виконує масивні обчислення. Оскільки вони мають ізольовані простори пам'яті, програміст повинен явно контролювати копіювання даних через шину PCIe.

Керування пам'яттю пристрою

Життєвий цикл даних на GPU

1. **cudaMalloc**: Виділення простору у відеопам'яті.
2. **cudaMemcpy**: Передача даних (HostToDevice).
3. **Kernel Launch**: Виконання обчислень.
4. **cudaMemcpy**: Повернення результатів (DeviceToHost).
5. **cudaFree**: Звільнення ресурсів.

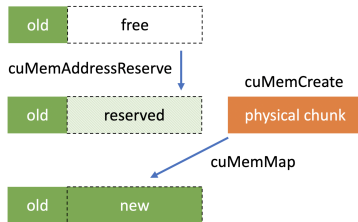


Рис. 3: Схема життєвого циклу даних

Написання ядра (Kernel)

Код ядра описує логіку для одного потоку. Специфікатор `__global__` вказує, що функція виконується на GPU.

Vector Addition (CUDA C)

```
__global__  
void vecAdd(float* A, float* B, float* C, int n) {  
    int i = blockDim.x * blockIdx.x + threadIdx.x;  
    if (i < n) {  
        C[i] = A[i] + B[i];  
    }  
}
```

Організація потоків (Grids, Blocks, Threads)

Потоки об'єднуються в блоки, а блоки – в сітку (Grid). Така ієрархія дозволяє CUDA працювати на будь-якому GPU, незалежно від кількості фізичних ядер.

Індексація:

$\text{index} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$

$\text{blockDim.x} + \text{threadIdx.x}$

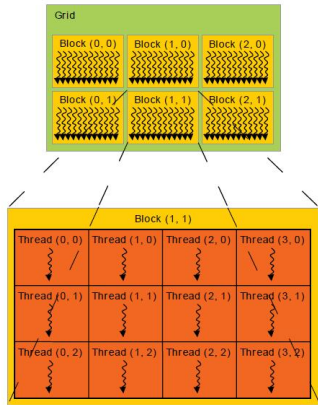


Рис. 4: Ієрархія потоків у CUDA

Rust та безпечні абстракції (Perspectives)

Використання Rust дозволяє замінити небезпечні вказівники та ручні індекси безпечними ітераторами.

CUDA C (Низький рівень):

```
// Ризик: out-of-bounds, memory leaks
cudaMalloc(&d_A, size);
vecAdd<<<grid, block>>>(d_A, ...);
```

Rust (Високий рівень):

```
// Безпечно: автоматичне керування пам'яттю
let c: Vec<_> = a.par_iter()
    .zip(&b)
    .map(|(x, y)| x + y)
    .collect();
```

Переваги Rust-підходу

Гарантія безпеки пам'яті (Memory Safety) на етапі компіляції та відсутність станів гонитви (Data Races).

Висновки

- CUDA C – це фундамент та повний контроль над залізом.
- Розуміння ієрархії потоків (SIMT) є критичним для роботи із паралельними даними.
- Майбутнє – за безпечними мовами (Rust) та високорівневими абстракціями (Rayon), що зменшують кількість помилок.